From String to Comparable

Interfaces, a comparable Counter, and generics

Marcus Birkenkrahe

COSC 2230 · CS III · Augustana University

Fall 2026 · Sedgewick & Wayne, *Introduction to Programming in Java* 2e, §4.2

By the end of this session you can

1. explain what a Java interface is: a promise to provide methods
2. turn the `String` insertion sort into one for any `Comparable` type
3. make your own class comparable with `implements Comparable<T>`
4. avoid the subtraction trap in `compareTo()`
5. read the "unchecked" warning and remove it with a generic method

The problem: one sort per type?

`Insertion.sort(String[] a)` sorts strings only.

How would you sort an `Integer[]`, a `Double[]`, a `Counter[]`?

Copy and replace

One copy per type – same algorithm, every bug fixed in every copy.

What the algorithm really needs

"For sorting objects in an array, we need only assume that we can compare two elements."

– book, p. 545

Interfaces: a promise

A Java **interface** lists methods without bodies. A class that **implements** it promises to provide them – the compiler checks.

```
public interface Comparable<Key>
{
    int compareTo(Key b);
}
```

Book p. 545: API for `java.util.Comparable`

- `String`, `Integer`, `Double` already implement it
- that is why `"eve".compareTo("oscar")` worked in binary search
- the order it defines: the type's **natural order**
- `<Key>`: both objects have the same type (see the end)

InsertionComparable

→ **Switch to editor · code-along 1**

1. Copy **`Insertion.java`** to **`InsertionComparable.java`**
2. Replace **`String`** by **`Comparable`** in `sort()` and `exch()` – nothing else changes
3. Test client: sort a `String[]`, an `Integer[]` and a `Double[]`
4. Compile – and read what the compiler says

Book Program 4.2.4 (p. 547) · classpath: `.:stdlib.jar`

One method, three types

```
public static void sort(Comparable[] a)
{
    int N = a.length;
    for (int i = 1; i < N; i++)
        for (int j = i; j > 0; j--)
            if (a[j-1].compareTo(a[j]) > 0)
                exch(a, j-1, j);
            else break;
}
```

```
[and, but, had, him, ...]
[3, 7, 19, 25, 42, 88]
[0.58, 1.41, 2.71, 3.14]
```

`int` is not an object: `Integer` is its **wrapper class**. Java converts automatically (**autoboxing**), so `{ 42, 7, 19 }` works.

The warning: a raw type

```
$ javac -Xlint:unchecked -cp .:stdlib.jar InsertionComparable.java
warning: [unchecked] unchecked call to compareTo(T) as a member
         of the raw type Comparable
```

`Comparable` without <. . .> is a **raw type**: the compiler cannot check that both objects have the same type. The book's program has the same warning.

What could go wrong?

```
Comparable[] a = { "eve", 42, "bob" };
InsertionComparable.sort(a);
```

```
ClassCastException: class
java.lang.Integer cannot be cast
to class java.lang.String
```

It compiles – and crashes at run time.

What compareTo() must guarantee

`a.compareTo(b)`: **negative**, **0** or **positive** – test the sign.

A total order (book, p. 546)

- **antisymmetric:** if $a \leq b$ and $b \leq a$, then $a = b$
- **transitive:** if $a \leq b$ and $b \leq c$, then $a \leq c$
- **total:** $a \leq b$ or $b \leq a$ – any two items can be compared

Otherwise the results are unpredictable: insertion sort trusts every compare.

```
error: incompatible types: Plain[] cannot be converted to Comparable[]
```

A class without the promise is rejected at compile time.

Your `Counter` from week 1, comparable

→ **Switch to editor · code-along 2**

1. Open **your `Counter.java`** from week 1
2. Add **`implements Comparable<Counter>`** to the class declaration
3. Write **`compareTo(Counter b)`**: compare the counts
4. Add **`toString()`** to print a counter as its count
5. Test client: N counters with random counts, sort them with `InsertionComparable`

Book p. 546 · files: `Counter.java` · `InsertionComparable.java`

A comparable Counter

```
public class Counter
             implements Comparable<Counter>
{
    private int count;
    ...
    public int compareTo(Counter b)
    {
        if      (count < b.count) return -1;
        else if (count > b.count) return +1;
        else                      return  0;
    }
    public String toString()
    { return "" + count; }
}
```

```
$ java Counter 10
[90, 80, 91, 74, 23, ...]
[23, 36, 43, 54, 74, ...]
```

b.count is private – why allowed?

→ Private to the class, not to the object.

Why does the old sort work?

→ It relies only on the promise.

Pitfall: compare by subtraction

A shortcut you will see online:

```
public int compareTo(Counter b) { return count - b.count; }
```

The sign is right ... until the subtraction overflows:

```
a = 2,000,000,000   b = -2,000,000,000
a - b                 = -294967296
Integer.compare(a, b) = 1
```

a is larger, but a − b is negative

The same trap as `(lo + hi) / 2` in binary search. Use `Integer.compare()` or the `if` chain.

Generics: the warning goes away

```
public static <Key extends Comparable<Key>> void sort(Key[] a)
```

`Key`: any type that is comparable with itself. No warning – and the mixed array is now caught by the compiler:

```
error: method sort in class InsertionGeneric cannot be applied
  required: Key[]   found: Object[]
```

Raw type

error at run time – maybe in front of a user

Generic

error at compile time – before the program runs

Next in §4.3: `Stack<Item>`, `Queue<Item>`, `ST<Key, Value>` – write the code once, let the compiler check the types.

Practice: a comparable Score

PRACTICE · 6 MIN

1. Write **`Score`**: a name and a number of points
2. Order by points; equal points by name
3. Use `Integer.compare()` and `String`'s `compareTo()`
4. Sort the data with **`InsertionGeneric`** and print it

Why do you need the tie-break on the name?

Data: `scores.txt`

```
wendy 72
alice 85
dave 72
walter 90
carlos 85
carol 60
```

Wrap-up: from String to Comparable

- An interface is a promise: `implements Comparable<T>` promises `compareTo()`
- One insertion sort for every comparable type
- `compareTo()`: negative, 0, positive; a total order; no subtraction
- Raw type: warning and run-time errors – generic: errors at compile time

Next (Monday): mergesort – the N log N sort.

Checkout question

You have 1,000 exam papers to sort by name, and two friends to help. How do you split the work?